AW Developer Guide Debian Docker CN

版本号: 1.0
发布日期: 2024.11.18

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2024.11.18	AWA1586	发布初版

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 介绍 **2**

3 内核配置 **3**

4 安装 **4**
4.1 构建时安装 4
4.2 运行时安装 4

5 使用 **6**
5.1 docker 环境检查 6
5.2 自建镜像测试 6
5.2.1 构建 7
5.2.2 测试 7
5.3 网络包测试 8
5.3.1 hello-world 8
5.3.2 ubuntu 9

6 FAQ **11**
6.1 Q1：dockerd 运行异常 11
6.2 Q2：网络连接失败 11
6.3 Q3：docker pull 失败，时间异常 12
6.4 Q4：目标机上 dockerd 启动过程，存在大量警告，并异常退出，无法运行 12

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 3-1　docker-check-config 3
图 4-1　docker-install 5
图 5-1　docker-version 6
图 5-2　docker-run-hello 8
图 5-3　docker-hello-world 9
图 6-1　dockerd-iptables-fail 11
图 6-2　iptables-legacy 11

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍基于 debian 文件系统的 Docker 模块的使用方法，方便开发人员使用。

1.2 目标读者

Docker 模块维护、使用人员。

1.3 适用范围

全志 AIOT 系列 Linux SDK(V1.3 版本及以上) debian 系统。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 介绍

Docker 是一个开源的应用容器引擎，开发者可以打包应用和依赖包到一个轻量级、可移植的容器中，然后发布到任何流行的 Linux 机器上，能够更高效的利用系统资源、保证一致的运行环境，实现持续交付和部署，以及后期更轻松的迁移、维护、扩展。

本文主要介绍 Docker 系统以下几项。

- docker 依赖的内核配置；
- 使用验证简述；
- 使用过程遇到的问题。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 内核配置

docker 依赖内核配置，具体参考官网链接：https://github.com/coreos/docker/blob/master/contrib/check-config.sh。

在目标机上输入以下命令可查看内核配置的匹配情况。

```
/usr/share/docker.io/contrib/check-config.sh
```

```
root@localhost:/test#
root@localhost:/test#
root@localhost:/test# /usr/share/docker.io/contrib/check-config.sh
info: reading kernel config from /proc/config.gz ...

Generally Necessary:
- cgroup hierarchy: cgroupv2
- CONFIG_NAMESPACES: enabled
- CONFIG_NET_NS: enabled
- CONFIG_PID_NS: enabled
- CONFIG_IPC_NS: enabled
- CONFIG_UTS_NS: enabled
- CONFIG_CGROUPS: enabled
- CONFIG_CGROUP_CPUACCT: enabled
- CONFIG_CGROUP_DEVICE: enabled
- CONFIG_CGROUP_FREEZER: enabled
- CONFIG_CGROUP_SCHED: enabled
- CONFIG_CPUSETS: enabled
- CONFIG_MEMCG: enabled
- CONFIG_KEYS: enabled
- CONFIG_VETH: enabled
- CONFIG_BRIDGE: enabled
- CONFIG_BRIDGE_NETFILTER: enabled
- CONFIG_IP_NF_FILTER: enabled
- CONFIG_IP_NF_TARGET_MASQUERADE: enabled
- CONFIG_NETFILTER_XT_MATCH_ADDRTYPE: enabled
- CONFIG_NETFILTER_XT_MATCH_CONNTRACK: enabled
- CONFIG_NETFILTER_XT_MATCH_IPVS: missing
- CONFIG_NETFILTER_XT_MARK: enabled
- CONFIG_IP_NF_NAT: enabled
- CONFIG_NF_NAT: enabled
- CONFIG_POSIX_MQUEUE: enabled

Optional Features:
```

图 3-1: docker-check-config

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 安装

一般 debian 系统会自带 docker。若无，可使用如下方式进行安装。

4.1 构建时安装

在具体方案的 **linaro.list.chroot** 文件新增 **docker.io**。这里给出一个例子，如下。

```
xxx@xx:${SDK_ROOT}/debian$ git diff
diff --git a/debian-build-service/bullseye-gnome-arm64/customization/package-lists/linaro.list.chroot b/debian-build-
        service/bullseye-gnome-arm64/customization/package-lists/linaro.list.chroot
index a38e6a4..be32acb 100644
--- a/debian-build-service/bullseye-gnome-arm64/customization/package-lists/linaro.list.chroot
+++ b/debian-build-service/bullseye-gnome-arm64/customization/package-lists/linaro.list.chroot
@@ -157,3 +157,6 @@ xinit
 xterm
 zathura
 guvcview
+
+#docker
+docker.io
```

此例子是在DEBIAN_TAR_ROOTFS=linaro-bullseye-xfce-arm64.tar.gz的方案增加 docker 软件包。

然后重新编译文件系统即可，编译过程保证网络畅通，具体编译请参考 debian 文件系统构建文档。编译后在 out 目录的具体方案下可找到 docker 的软件包。

```
xxx@xx:${SDK_ROOT}/out/${LICHEE_IC}/${LICHEE_BOARD}/debian/rootfs_def$ find -name "docker"
./usr/bin/docker
./usr/share/bash-completion/completions/docker
./etc/docker
./etc/default/docker
./etc/init.d/docker
```

4.2 运行时安装

```
sudo apt install docker.io
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@localhost:~#
root@localhost:~# sudo apt install docker.io
sudo: unable to resolve host localhost.localdomain: Name or service not known
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (20.10.5+dfsg1-1+deb11u3).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@localhost:~#
```

图 4-1: docker-install

上图是显示 docker 已经安装了，版本为20.10.5+dfsg1-1+deb11u3。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 使用

5.1 docker 环境检查

通过docker verison 或者docker info可查看相关信息。

```
root@localhost:/test#
root@localhost:/test#
root@localhost:/test# docker version
Client:
 Version:           20.10.5+dfsg1
 API version:       1.41
 Go version:        go1.15.15
 Git commit:        55c4c88
 Built:             Sun Oct 13 16:05:55 2024
 OS/Arch:           linux/arm64
 Context:           default
 Experimental:      true

Server:
 Engine:
  Version:          20.10.5+dfsg1
  API version:      1.41 (minimum version 1.12)
  Go version:       go1.15.15
  Git commit:       363e9a8
  Built:            Sun Oct 13 16:05:55 2024
  OS/Arch:          linux/arm64
  Experimental:     false
 containerd:
  Version:          1.4.13~ds1
  GitCommit:        1.4.13~ds1-1~deb11u4
 runc:
  Version:          1.0.0~rc93+ds1
  GitCommit:        1.0.0~rc93+ds1-5+deb11u5
 docker-init:
  Version:          0.19.0
  GitCommit:
root@localhost:/test#
```

图 5-1: docker-version

5.2 自建镜像测试

本 demo 创建一个最小的 awhello 镜像并运行，来确保运行环境的正确性。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5.2.1 构建

1. 源代码

```
#include <sys/syscall.h>

void _start() {
   syscall(SYS_write, 1, "hello AW\n", sizeof("hello AW\n") - 1);
   syscall(SYS_exit, 0);
}
```

2. build

```
xxx-gcc -static -Os -nostartfiles -fno-asynchronous-unwind-tables -o hello hello.c
```

3. Dockerfile

```
FROM scratch
MAINTAINER AW aw@allwinnertechtech.com
COPY hello /
CMD ["/hello"]
```

5.2.2 测试

把对应的二进制程序 **hello** 和 **Dockerfile** 拷贝到目标机 **test**（新建）目录下。然后进行如下操作。

```
root@localhost:~/test# ls -ls
total 16
 4 -rwxr-xr-x 1 root root   80 Nov 18  2024 Dockerfile
12 -rwxr-xr-x 1 root root 11024 Nov 18  2024 hello

# 构建版本为1.0的awhello的docker镜像
root@localhost:~/test# docker build -t awhello:1.0 .

# 查看当前镜像列表
root@localhost:~/test# docker image ls

# 运行1.0的awhello镜像
root@localhost:~/test# docker run awhello:1.0
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@localhost:~/test#
root@localhost:~/test#
root@localhost:~/test#
root@localhost:~/test# docker build -t awhello:1.0 .
Sending build context to Docker daemon  13.82kB
Step 1/4 : FROM scratch
 --->
Step 2/4 : MAINTAINER AW aw@allwinnertechtech.com
 ---> Running in c968806cc641
Removing intermediate container c968806cc641
 ---> 152df2fb1123
Step 3/4 : COPY hello /
 ---> 51de51e8c571
Step 4/4 : CMD ["/hello"]
 ---> Running in d903a3b99973
Removing intermediate container d903a3b99973
 ---> 438c1357e8f5
Successfully built 438c1357e8f5
Successfully tagged awhello:1.0
root@localhost:~/test# docker image ls
REPOSITORY   TAG       IMAGE ID       CREATED          SIZE
awhello      1.0       438c1357e8f5   14 seconds ago   11kB
root@localhost:~/test#
root@localhost:~/test#
root@localhost:~/test# docker run awhello:1.0
[  414.190631] docker0: port 1(veth92b097d) entered blocking state
[  414.197335] docker0: port 1(veth92b097d) entered disabled state
[  414.204133] device veth92b097d entered promiscuous mode
[  414.847664] eth0: renamed from vethdb69818
[  414.895831] IPv6: ADDRCONF(NETDEV_CHANGE): veth92b097d: link becomes ready
[  414.903729] docker0: port 1(veth92b097d) entered blocking state
[  414.910365] docker0: port 1(veth92b097d) entered forwarding state
[  414.917342] IPv6: ADDRCONF(NETDEV_CHANGE): docker0: link becomes ready
hello AW
[  415.135364] docker0: port 1(veth92b097d) entered disabled state
[  415.142124] vethdb69818: renamed from eth0
[  415.217693] docker0: port 1(veth92b097d) entered disabled state
[  415.225050] device veth92b097d left promiscuous mode
[  415.230638] docker0: port 1(veth92b097d) entered disabled state
root@localhost:~/test#
```

图 5-2: docker-run-hello

5.3 网络包测试

5.3.1 hello-world

```
#使用docker下载、运行网络仓库的的 hello-world.
docker run hello-world
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
daemon.json  key.json
root@localhost:/etc/docker# docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
478afc919002: Pull complete
Digest: sha256:305243c734571da2d100c8c8b3c3167a098cab6049c9a5b066b6021a60fcb966
Status: Downloaded newer image for hello-world:latest
[ 3784.347017] docker0: port 1(vetha98fe3d) entered blocking state
[ 3784.353705] docker0: port 1(vetha98fe3d) entered disabled state
[ 3784.360547] device vetha98fe3d entered promiscuous mode
[ 3784.863845] eth0: renamed from veth8b902fc
[ 3784.894864] IPv6: ADDRCONF(NETDEV_CHANGE): vetha98fe3d: link becomes ready
[ 3784.902760] docker0: port 1(vetha98fe3d) entered blocking state
[ 3784.909428] docker0: port 1(vetha98fe3d) entered forwarding state

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (arm64v8)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples[ 3785.078229] docker0: port 1(vetha98fe3d) entered disabled state
 and ideas, visit:
 https://doc[ 3785.085387] veth8b902fc: renamed from eth0
s.docker.com/get-started/

[ 3785.175380] docker0: port 1(vetha98fe3d) entered disabled state
[ 3785.182689] device vetha98fe3d left promiscuous mode
[ 3785.188313] docker0: port 1(vetha98fe3d) entered disabled state
root@localhost:/etc/docker# [ 3789.615035] >>> rwnx_cfg80211_scan()
```

图 5-3: docker-hello-world

5.3.2 ubuntu

通过 docker 的两个参数 -i -t，让 docker 运行的容器实现 **“对话”** 的能力。

```
# 下载最新的ubuntu镜像
root@localhost:/# docker pull ubuntu

# 查看镜像是否已经下载完成
root@localhost:/# docker images
REPOSITORY   TAG      IMAGE ID      CREATED       SIZE
ubuntu       latest   2d4e6732b03f  4 weeks ago   101MB  # 目标镜像
awhello      1.0      438c1357e8f5  2 months ago  11kB
hello-world  latest   ee301c921b8a  18 months ago  9.14kB
root@localhost:/#

# 运行ubuntu镜像
docker run -i -t ubuntu /bin/bash
root@localhost:/# docker run -i -t ubuntu /bin/bash
[ 4714.515614] docker0: port 1(vethb0e2664) entered blocking state
[ 4714.522321] docker0: port 1(vethb0e2664) entered disabled state
[ 4714.529152] device vethb0e2664 entered promiscuous mode
[ 4715.063770] eth0: renamed from veth272b87a
[ 4715.095872] IPv6: ADDRCONF(NETDEV_CHANGE): vethb0e2664: link becomes ready
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
[ 4715.103767] docker0: port 1(vethb0e2664) entered blocking state
[ 4715.110409] docker0: port 1(vethb0e2664) entered forwarding state
[ 4716.027739] >>> rwnx_send_scanu_req()
root@f31f1b4bffda:/# ls
bin  dev  home  media  opt  root  sbin  sys  usr
boot  etc  lib  mnt  proc  run  srv  tmp  var
root@f31f1b4bffda:/# exit
exit
[ 4865.634639] docker0: port 1(vethb0e2664) entered disabled state
[ 4865.641395] veth272b87a: renamed from eth0
[ 4865.729577] docker0: port 1(vethb0e2664) entered disabled state
[ 4865.736942] device vethb0e2664 left promiscuous mode
[ 4865.742538] docker0: port 1(vethb0e2664) entered disabled state
root@localhost:/#
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 Q1：dockerd 运行异常

```
root@localhost:~/test#
root@localhost:~/test# dockerd
INFO[2024-08-25T22:14:52.161477133Z] Starting up
INFO[2024-08-25T22:14:52.164588050Z] parsed scheme: "unix"                         module=grpc
INFO[2024-08-25T22:14:52.164691425Z] scheme "unix" not registered, fallback to default scheme  module=grpc
INFO[2024-08-25T22:14:52.164787758Z] ccResolverWrapper: sending update to cc: {[{unix:///run/containerd/containerd.sock  <nil> 0 <nil>}] <nil> <nil>}  module=grpc
INFO[2024-08-25T22:14:52.164830800Z] ClientConn switching balancer to "pick_first"  module=grpc
INFO[2024-08-25T22:14:52.169229425Z] parsed scheme: "unix"                         module=grpc
INFO[2024-08-25T22:14:52.169305883Z] scheme "unix" not registered, fallback to default scheme  module=grpc
INFO[2024-08-25T22:14:52.169589925Z] ccResolverWrapper: sending update to cc: {[{unix:///run/containerd/containerd.sock  <nil> 0 <nil>}] <nil> <nil>}  module=grpc
INFO[2024-08-25T22:14:52.169694883Z] ClientConn switching balancer to "pick_first"  module=grpc
INFO[2024-08-25T22:14:52.183182175Z] [graphdriver] using prior storage driver: overlay2
WARN[2024-08-25T22:14:52.186665216Z] Unable to find io controller
WARN[2024-08-25T22:14:52.186744216Z] Unable to find pids controller
INFO[2024-08-25T22:14:52.187259466Z] Loading containers: start.
INFO[2024-08-25T22:14:52.365589008Z] stopping event stream following graceful shutdown  error="<nil>" module=libcontainerd namespace=moby
failed to start daemon: Error initializing network controller: error obtaining controller instance: failed to create NAT chain DOCKER: iptables failed: iptables --wait -t nat -N DOCKER:
iptables v1.8.7 (nf_tables):  TABLE_ADD failed (Operation not supported): table nat
 (exit status 4)
root@localhost:~/test#
```

图 6-1: dockerd-iptables-fail

经排查，docker 依赖 iptables-legacy，debian 系统如果包含 iptables-nft，会优先使用 iptables-nft。需要切回 iptables-legacy。如下。

```
update-alternatives --set iptables /usr/sbin/iptables-legacy
```

```
root@localhost:~#
root@localhost:~# update-alternatives --set iptables /usr/sbin/iptables-legacy
update-alternatives: using /usr/sbin/iptables-legacy to provide /usr/sbin/iptables (iptables) in manual mode
root@localhost:~# iptables
iptables v1.8.7 (legacy): no command specified
Try `iptables -h' or 'iptables --help' for more information.
root@localhost:~# systemctl restart docker
root@localhost:~#
```

图 6-2: iptables-legacy

6.2 Q2：网络连接失败

```
Error response from daemon: Get "https://registry-1.docker.io/v2/": net/http: request canceled while waiting for
        connection (Client.Timeout exceeded while awaiting headers)
```

1. 检查 docker 服务是否正常；
2. 检查网络是否异常；
3. 增大 Docker 超时时间；
4. 更换国内可用的镜像。目前国内很多都不可用。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@localhost:/etc/docker# cat daemon.json
{
  "registry-mirrors": ["https://<your-mirror>.com"],
  "http-timeout": 300
}
```

6.3 Q3：docker pull 失败，时间异常

```
Unable to find image 'hello-world:latest' locally
docker: Error response from daemon: Get "https://registry-1.docker.io/v2/": x509: certificate has expired or is not yet
        valid: current time 2024-08-25T19:53:49Z is before 2024-09-02T00:00:00Z.
See 'docker run --help'.
```

更新系统时间即可。

6.4 Q4：目标机上 dockerd 启动过程，存在大量警告，并异常退出，无法运行

docker 需要内核开启 cgroups、namespace、netfilter、overlayfs 等配置支持，才能正常运行，可以通过运行 /usr/share/docker.io/contrib/check-config.sh 脚本检查当前系统内核的配置是否满足 docker 运行的要求。

```
/usr/share/docker.io/contrib/check-config.sh
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2024 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。